

DESIGN AND ANALYSIS OF ALGORITHMS

Coding Challenge Task - Q7

LONGEST COMMON SUBSEQUENCE (LCS)

Dynamic Programming vs Greedy Matching

Problem	Find the LCS of ABCBDAB and BDCAB
DP result	BCAB, length 4
Greedy result	AB, length 2
Core finding	Greedy finds a valid common subsequence, but not the maximum-length one

Report flow: experimental setup -> input -> algorithms -> DP table -> greedy trace -> program output -> comparison -> correctness -> complexity -> optimal substructure -> rubric -> submission structure.

1. Experimental Setup

Item	Value
First string X	ABCB DAB
Second string Y	BDCAB
Language	Python 3
Task	Compute an LCS and compare it with a greedy matching attempt

An LCS is a sequence that occurs in both strings in the same relative order. Characters need not be adjacent. The objective is to maximize its length.

2. Input

Exact input used for execution:

```
Enter first string: ABCBDAB
Enter second string: BDCAB
```

3. Method A - Dynamic Programming

Define $dp[i][j]$ as the LCS length of $X[:i]$ and $Y[:j]$. If $X[i-1]$ equals $Y[j-1]$, extend the smaller subproblem. Otherwise keep the better of the two prefix states.

Recurrence

```
dp[i][j] = dp[i-1][j-1] + 1, if X[i-1] = Y[j-1]
dp[i][j] = max(dp[i-1][j], dp[i][j-1]), otherwise
```

4. Method B - Greedy Matching Attempt

Scan X from left to right. For each character, choose its first occurrence in Y after the current search position and never reconsider the choice. This is a deliberate greedy baseline for demonstrating failure, not a correct general LCS algorithm.

5. DP Table and Optimal Result

The completed table gives $dp[7][5] = 4$. Backtracking one optimal path reconstructs BCAB.

X \ Y	-	B	D	C	A	B
-	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3
D	0	1	2	2	2	3
A	0	1	2	2	3	3
B	0	1	2	2	3	4

Dynamic Programming result: LCS = BCAB, length = 4.

6. Greedy Trace

X character	Search / pointer movement in Y	Decision	Current result
A	Skip B,D,C; match A at Y[3]	Select A	A
B	Continue from Y[4]; match B	Select B	AB
C	No positions remain	Stop	AB

The greedy method first chooses A at Y[3], then can still choose B at Y[4]. It returns AB, which is a valid common subsequence of length 2, but it misses the optimal BCAB of length 4.

7. Program Input and Output

Input

```
ABCBDA  
BDCAB
```

Output

```
Enter first string: Enter second string:  
Dynamic Programming  
LCS = BCAB  
LCS Length = 4
```

```
Greedy Matching Attempt  
Greedy Result = AB  
Greedy Length = 2
```

```
Conclusion: Greedy failed to find the optimal LCS.
```

8. Result Comparison

Measure	Dynamic Programming	Greedy Attempt
Subsequence	BCAB	AB
Length	4	2
Optimal	Yes	No
General correctness	Guaranteed for LCS	No guarantee

9. Why Greedy Fails

LCS does not have the greedy-choice property required for a generally correct greedy algorithm. A locally early match can consume positions that would be needed for a longer later sequence. Here, choosing A first leads to AB, while an alternative start with B allows the length-4 solution BCAB.

Optimal substructure

An optimal solution for a larger pair of prefixes can be expressed using optimal solutions of smaller prefix pairs. This supports the DP recurrence.

Overlapping subproblems

Many prefix-pair LCS calculations repeat. The DP table stores each result once, avoiding exponential recomputation of the naive recursive formulation.

10. Complexity Analysis

Method	Time	Space	Guarantee
Dynamic Programming	$O(mn)$	$O(mn)$	Optimal LCS
Greedy pointer scan	$O(m+n)$	$O(1)$ excluding output	No optimality guarantee

The greedy baseline is faster for this pointer-scan implementation, but the speed advantage does not establish correctness. The DP solution is preferred when an optimal LCS is required.

11. Complete Python Program

```
def lcs_dp(x, y):
    """Return one LCS and its length using Dynamic Programming."""
    m = len(x)
    n = len(y)

    # dp[i][j] = LCS length of x[:i] and y[:j]
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i - 1] == y[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    # Reconstruct one optimal LCS
    i, j = m, n
    result = []

    while i > 0 and j > 0:
        if x[i - 1] == y[j - 1]:
            result.append(x[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1

    result.reverse()
    return "".join(result), dp[m][n]

def greedy_lcs_attempt(x, y):
    """
    Greedy attempt:
    Scan x from left to right and always choose the first
    matching character available in y after the current position.
    """
    j = 0
    result = []

    for ch in x:
        while j < len(y) and y[j] != ch:
            j += 1

        if j == len(y):
            break

        result.append(ch)
        j += 1

    return "".join(result)

# Input
x = input("Enter first string: ").strip()
y = input("Enter second string: ").strip()

lcs_result, lcs_length = lcs_dp(x, y)
greedy_result = greedy_lcs_attempt(x, y)
```

```

print("\nDynamic Programming")
print("LCS =", lcs_result)
print("LCS Length =", lcs_length)

print("\nGreedy Matching Attempt")
print("Greedy Result =", greedy_result)
print("Greedy Length =", len(greedy_result))

if len(greedy_result) == lcs_length:
    print("\nConclusion: Greedy found an optimal LCS for this input.")
else:
    print("\nConclusion: Greedy failed to find the optimal LCS.")

```

12. Rubric-Oriented Evaluation

Criterion	Evidence
Problem Understanding - 20%	Defines LCS, states the exact strings, and explains the comparison and correctness requirement.
Algorithm - 25%	Uses the standard DP recurrence and reconstruction, plus an explicit greedy baseline.
Code Implementation - 20%	Modular Python functions, input handling, table-based LCS reconstruction, and clear output.
Competitive Performance - 20%	DP runs in $O(mn)$, while the DP table prevents repeated subproblem computation.
Test Case Handling - 15%	Executes the required input and reports both DP and greedy results for direct comparison.

13. GitHub Submission Structure

```

CO4_AT1/
├── Question_1/
│   ├── Source_Code/
│   │   └── lcs_vs_greedy.py
│   └── Report/
│       └── Q7_LCS_vs_Greedy_Report.pdf
└── README.md

```

Include screenshots of the source code and actual program execution in the report materials before the final GitHub submission.

14. Conclusion

For ABCBDAB and BDCAB, Dynamic Programming finds the optimal LCS BCAB of length 4. The selected greedy attempt returns AB of length 2. The result demonstrates that LCS cannot generally be solved by a simple greedy choice because a local match can prevent a better global subsequence. Optimal substructure and overlapping subproblems make Dynamic Programming the appropriate general technique.